ICMC USP SÃO CARLOS

USP Universidade de São Paulo



IA, Otimização e IoT para Indústrias e Negócios
Do Zero ao MLOps

PROFESSOR

Dr. Cláudio Fabiano Motta Toledo

OpAIoT
Optimization • AI • IoT

Módulo 1

Fundamentos de Dados e Aprendizado de Máquina

Aplicado a Operações e Negócios

OpAIoT
Optimization • AI • IoT

Universidade de São Paulo

Transformação Numérico Simbólico

Sumário

01 O que é Transformação Numérico Simbólico?

02 Intervalos de mesma largura

03 Intervalos de mesma frequência

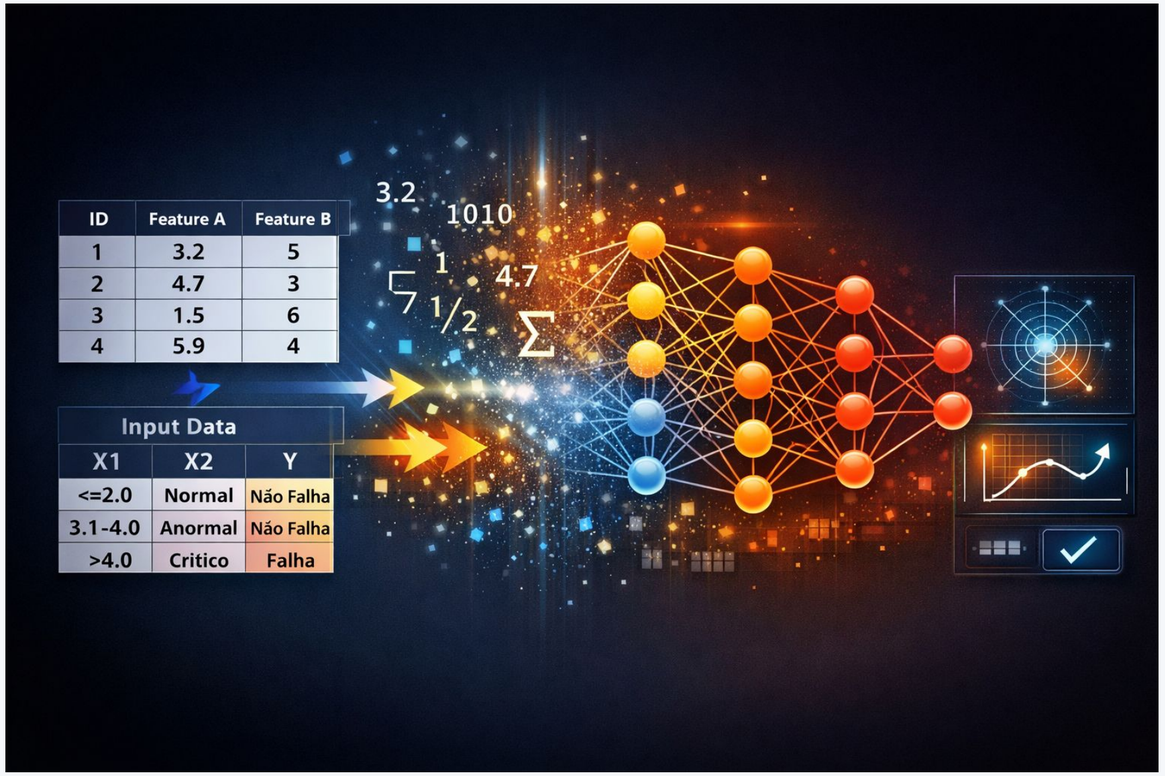
04 Discretização baseada em agrupamento

O que é Transformação Numérico Simbólica ?

Definição: a transformação numérico-simbólica normalmente significa converter um atributo numérico contínuo e**m categorias, faixas ou rótulos simbólicos**.

Por que isso é tão importante?

- Trabalhar com faixas é mais **interpretável** ou alinhado ao conhecimento de negócio, ajudando o modelo a capturar relações não lineares.
  - Cria uma ponte entre números e símbolos para que máquinas possam raciocinar sobre conceitos abstratos a partir de dados bruto, manipulando símbolos de forma mais flexível e robusta.



Atributos Discreto e Binário: Deve-se apenas associar um nome a cada valor.

Se o atributo original for uma sequência binária sem uma relação de ordem entre si, deve-se substituir por um nome ou categoria.

Podemos simplesmente decodificar uma das codificações propostas anteriormente, por exemplo, **one hot encoding**.



- Métodos de discretização são aplicados nos demais casos:
  - Intervalos de mesma largura (equal-width binning)
  - intervalos de mesma frequência (equal-frequency binning ou quantile binning)
  - Discretização baseada em agrupamento

Transforma uma variável numérica contínua em um conjunto de faixas categóricas, todas com a **mesma amplitude.**

Ideia básica

1. **Suponha uma variável com valor mínimo min e máximo max para criação de k faixas.**
2. **Largura de cada faixa:**

   L= (max-min)/k

O intervalo total fica dividido em k partes consecutivas com mesma largura.

Cliente	Idade (numérica)	Faixa etária (simbólica)
C1	19	18–38
C2	22	18–38
C3	27	18–38
C4	35	18–38
C5	38	18–38
C6	41	38–58
C7	46	38–58
C8	52	38–58
C9	57	38–58
C10	61	58–78
C11	69	58–78
C12	75	58–78

Exemplo: Temos 3 bins de mesma largura

L = (78-18)/3= 20 anos

com faixas:

18–38, 38–58 e 58–78

18–38 :
38–58 :
58–78 :



Intervalos de mesma largura (equal-width binning)

Vantagem:

- Fácil de implementar.
- Facilita a visualização e comunicação dos dados.
- Útil quando limites das faixas fazem sentido no domínio.
- Transforma relações não lineares em algo mais tratável por alguns modelos.

Desvantagem:

- Não considera a distribuição dos dados.
  - Pode gerar bins cheios e outros quase vazios.
- Valores extremo influenciam a largura.
- Perda de informação trocando contínuos por faixas.
- A escolha do número de bins pode ser arbitrária.
- Nem sempre melhora o desempenho preditivo.

No pandas, cut() segmenta valores contínuos.

em intervalos

```python
import pandas as pd

idades = [18, 22, 35, 41, 57, 63, 78]

faixas = pd.cut(idades, bins=3)
print(faixas)
```

No scikit-learn, KBinsDiscretizer(strategy="uniform")

implementa a discretização por largura constante.

```python
from sklearn.preprocessing import KBinsDiscretizer
import numpy as np

X = np.array([[18], [22], [35], [41], [57], [63], [78]])

disc = KBinsDiscretizer(n_bins=3, encode='ordinal', strategy='uniform')
X_disc = disc.fit_transform(X)
```

Intervalos de mesma frequência (equal-frequency binning ou quantile binning)

Transforma uma variável numérica contínua em faixas categóricas de modo que cada faixa contenha aproximadamente o **mesmo número de observações.**

Ideia básica

1. ordenar os valores da variável;
2. escolher o número de bins k;
3. dividir a lista ordenada em k grupos com quantidades parecidas de elementos;
4. usar os pontos de corte entre esses grupos como limites das faixas.

Exemplo: Gasto mensal.

Criamos k=4 bins com aproximadamente 3 observações e larguras distintas.

Cliente	gasto_mensal	gasto_faixa
C1	12	Bin 1 (12–18)
C2	15	Bin 1 (12–18)
C3	18	Bin 1 (12–18)
C4	20	Bin 2 (20–25)
C5	22	Bin 2 (20–25)
C6	25	Bin 2 (20–25)
C7	28	Bin 3 (28–40)
C8	35	Bin 3 (28–40)
C9	40	Bin 3 (28–40)
C10	55	Bin 4 (55–120)
C11	70	Bin 4 (55–120)
C12	120	Bin 4 (55–120)

Intervalos de mesma frequência (equal-frequency binning ou quantile binning)

Transforma uma variável numérica contínua em faixas categóricas de modo que cada faixa contenha aproximadamente o **mesmo número de observações**

Exemplo: Gasto mensal.

Criamos k=4 bins com aproximadamente 3 observações e larguras distintas.

Cliente	gasto_mensal	gasto_faixa
C1	12	Bin 1 (12–18)
C2	15	Bin 1 (12–18)
C3	18	Bin 1 (12–18)
C4	20	Bin 2 (20–25)
C5	22	Bin 2 (20–25)
C6	25	Bin 2 (20–25)
C7	28	Bin 3 (28–40)
C8	35	Bin 3 (28–40)
C9	40	Bin 3 (28–40)
C10	55	Bin 4 (55–120)
C11	70	Bin 4 (55–120)
C12	120	Bin 4 (55–120)

Bin	Valores	Nº de observações	Largura
Bin 1	[12, 15, 18]	3	6
Bin 2	[20, 22, 25]	3	5
Bin 3	[28, 35, 40]	3	12
Bin 4	[55, 70, 120]	3	65

Intervalos de mesma frequência (equal-frequency binning ou quantile binning)

Vantagem:

- Bins com quantidades mais equilibrados.
- Natural dividir em quartis, decis e percentis.
- Adequado quando a **distribuição é muito desigual,** pois cortes acompanham a ordenação ao invés de larguras fixas.

Desvantagem:

- **Intervalos com larguras numéricas diferentes**, dificultando interpretação em alguns contextos.
- Por ser não supervisionado, não usa variável-alvo para escolher cortes; podendo perder informação útil.
- Em dados com muitos valores repetidos, podem surgir bordas duplicadas

O recurso mais direto em Python é pandas.qcut(). A documentação oficial define qcut() exatamente como uma função de discretização baseada em quantis.

```
gastos = pd.Series([12, 15, 18, 20, 22, 25, 28, 35, 40, 55, 70, 120])
faixas_rotuladas = pd.qcut(
    gastos,
    q=4,
    labels=["Q1 - baixo", "Q2 - médio-baixo",
            "Q3 - médio-alto", "Q4 - alto"]
)

faixas_rotuladas.value_counts().sort_index()
```

No sckikit-learn, temos o KBinsDiscretizer() onde a estratégia quantile cria bins com o mesmo número de pontos.

```
X = np.array([[12], [15], [18], [20], [22], [25], [28],
              [35], [40], [55], [70], [120]])

disc = KBinsDiscretizer(
    n_bins=4,
    encode="ordinal",       # retorna 0,1,2,3
    strategy="quantile"     # bins de mesma frequência
)

X_disc = disc.fit_transform(X)

print("Valores discretizados:")
print(X_disc.ravel())

print("\nLimites dos bins:")
print(disc.bin_edges_[0])
```

Intervalos de mesma frequência (equal-frequency binning ou quantile binning)

Exemplo. Numa análise de risco,valores até R$ 2.000 indicam risco de inadimplência muito alto, entre R$ 2.001 e R$ 8.000, risco baixo e relativamente estável, enquanto acima de R$ 8.000, o risco seria muito baixo.

Os pontos realmente importantes para prever a variável-alvo seriam algo como R$ 2.000 e R$ 8.000, porque neles o comportamento da inadimplência muda.

Aplicando intervalos de mesma frequência em 3 faixas, temos::

Faixa 1: até R$ 3.500

Faixa 2: de R$ 3.501 até R$ 6.500

Faixa 3: acima de R$ 6.500

Perceba o problema:

- Faixa 1: mistura pessoas com renda muito baixa e baixa-média, embora o risco mude bastante perto de R$ 2.000.
- Faixa 3: junta rendas médias-altas e muito altas, escondendo o fato de que acima de R$ 8.000 o risco cai ainda mais.

Ou seja, os cortes ficaram bons para equilibrar a quantidade de exemplos em cada grupo, mas não ficaram bons para separar o comportamento da variável-alvo. Assim, parte da informação preditiva da renda foi perdida.

Transforma uma variável numérica contínua em intervalos discretos definidos a partir de agrupamentos dos próprios dados, em vez de usar cortes de mesma largura ou mesma frequência.

Ideia básica

- Não divide o eixo numérico por largura fixa, mas por faixas de concentração do dados.
- Os limites entre bins surgem a partir dos grupos encontrados.
- Intervalos ficam mais aderentes à estrutura empírica dos dados em distribuições irregulares, multimodais, ou com regiões densas e regiões esparsas.

Passos

1. Escolhe-se o número de bins k
2. Aplica-se **K-Means** aos valores do atributo (univariado).
3. Obtém-se k centróides.
4. Limites entre bins colocados entre centróides vizinhos.

Exemplo numérico: Considere os valores

x=[1,2,3,4,10,11,12,20,21,22].

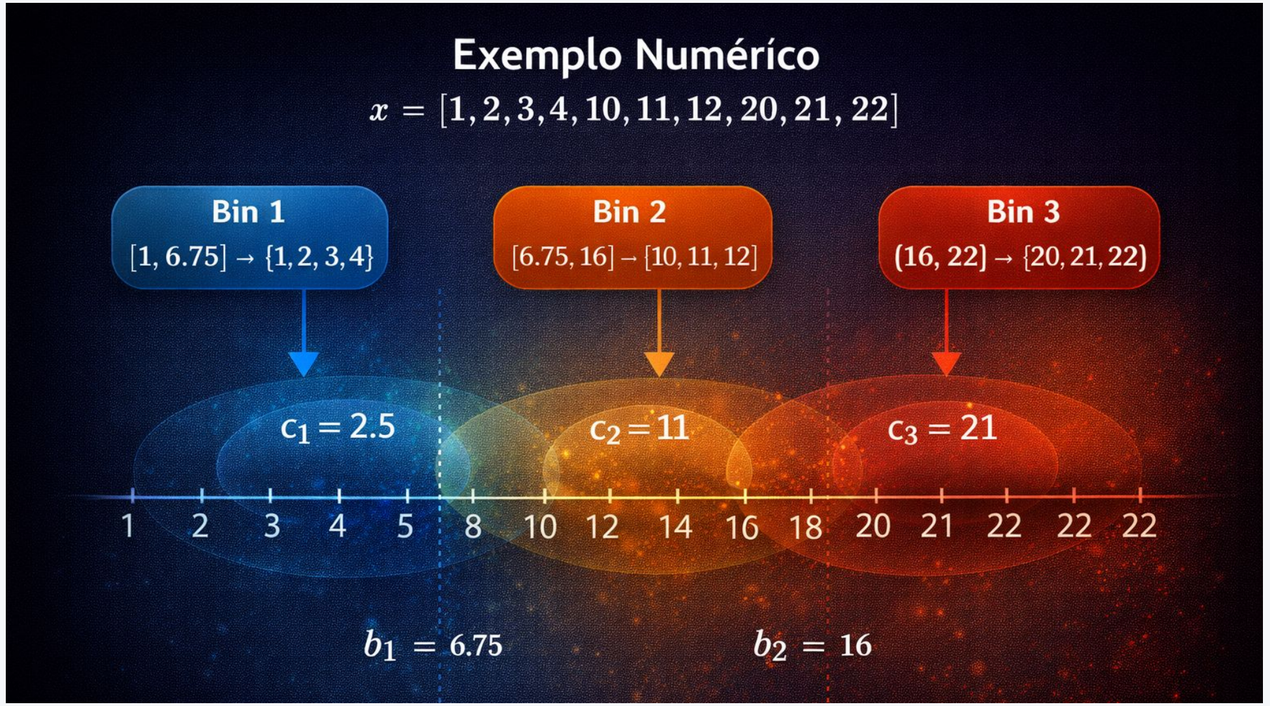
Se escolhemos k=3, uma solução plausível do k-means é formar três grupos aproximadamente centrados em:

$c1$=2.5, $c2$=11. $c3$=21

Os limites entre bins podem então ser tomados pelos pontos médios:

$b1$=(2.5+11)/2=6.75

$b2$=(11+21)/2=16



Observe que esses intervalos não têm a mesma largura e não têm necessariamente a mesma quantidade de pontos. Eles são definidos pela estrutura dos agrupamentos, não por uma regra uniforme ou por quantis.

Vantagem:

- Adapta-se melhor à distribuição dos dados..
- Pode produzir intervalos mais semânticos em dados multimodais.
- É uma alternativa prática e disponível em bibliotecas amplamente usadas.
  - O K-Means costuma ser rápido na prática.

Desvantagem:

- É preciso escolher o número de bins.
- K-Means pode cair em mínimos locais, demandando reinicializações
- Pode ser mais custosa do que regras muito simples como equal-width.

```
# Exemplo simples
X = np.array([[1], [2], [3], [4], [10], [11], [12], [20], [21], [22]])

disc = KBinsDiscretizer(
    n_bins=3,
    encode="ordinal",    # retorna 0, 1, 2
    strategy="kmeans"    # discretização baseada em agrupamento
)

X_disc = disc.fit_transform(X)

print("Bins atribuídos:")
print(X_disc.ravel())

print("Limites dos bins:")
print(disc.bin_edges_[0])
```

- n_bins=3 define o número de intervalos;
- s**trategy="kmeans"** ativa a discretização baseada em agrupamento;
  encode="ordinal" devolve os rótulos 0, 1 e 2 para os bins.

No scikit-learn, as três estratégias padrão são:

- **uniform**: bins com larguras idênticas;
- **quantile**: bins com aproximadamente o mesmo número de amostras;
- **kmeans:** bins definidos pelos centróides do K-Means.

OpAIoT
Optimization • AI • IoT

Universidade de São Paulo

Referências

- Faceli, Katti, et al. "Inteligência artificial: uma abordagem de aprendizado de máquina." (2021).
- Pandas User Guide. Discretization and quantiling. https://pandas.pydata.org/pandas-docs/stable/user_guide/basics.html





Obrigado!

opaiot@icmc.usp.br
+55 (16) 99962-7637

PROFESSOR

Dr. Cláudio Fabiano Motta Toledo

OpAIoT
Optimization • AI • IoT